

High-Level Design (HLD)

RAG Customer Support Assistant
with LangGraph & Human-in-the-Loop (HITL)

Field	Value
Document Type	High-Level Design (HLD)
Project	RAG Customer Support Assistant
Technology	LangGraph, ChromaDB, HuggingFace, OpenAI
Status	Final

Table of Contents

1. System Overview
2. Architecture Diagram
3. Component Descriptions
4. Data Flow
5. Technology Choices
6. Scalability Considerations

1. System Overview

1.1 Problem Definition

Traditional customer support systems rely on keyword search or static FAQ databases that fail to handle nuanced, context-dependent queries. When the knowledge base grows, precision degrades and customers receive irrelevant or incomplete answers. Additionally, complex cases that require human judgment are either handled poorly by automated systems or are never escalated at the right time.

1.2 Proposed Solution

This system implements a Retrieval-Augmented Generation (RAG) pipeline combined with a graph-based workflow engine (LangGraph) to deliver accurate, context-grounded answers. Whenever the system's confidence is low or the query matches escalation criteria, a Human-in-the-Loop (HITL) module automatically routes the ticket to a human agent.

1.3 Scope

- PDF-based knowledge base ingestion and semantic indexing
- Conversational query answering using LLM with retrieved context
- Intent-aware conditional routing (auto-answer vs. escalate)
- HITL escalation ticket creation and response integration
- CLI interface with demo, ingest, chat, and query commands
- Extensible to web front-end, multi-document support, and feedback loops

3. Component Descriptions

Component	Location	Responsibility
Document Loader	src/ingestion.py	Uses LangChain's PyPDFLoader. Retains page number and source file as metadata.
Chunking Module	src/ingestion.py	Applies RecursiveCharacterTextSplitter (size=500, overlap=100).
Embedding Model	src/ingestion.py	HuggingFace all-MiniLM-L6-v2. Runs on CPU, no API cost.
Vector Store	chroma_db/	ChromaDB persists embeddings to disk. Supports similarity search.

Retriever	src/retrieval.py	Queries ChromaDB for top-4 chunks. Computes a low-confidence flag if score < 0.6.
LLM	src/llm.py	OpenAI GPT-3.5-turbo. Enforces context-only answering via prompts.
Graph Workflow Engine	src/workflow.py	LangGraph StateGraph with 6 nodes. State flows across nodes.
Routing Layer	src/workflow.py	Conditional routing based on confidence, keywords, and LLM uncertainty.

4. Data Flow

4.1 Ingestion Pipeline

- Step 1: PDF File → Raw PDF loaded by PyPDFLoader
- Step 2: Page Documents → RecursiveCharacterTextSplitter → N chunks
- Step 3: Text Chunks → HuggingFace embedder encodes each chunk
- Step 4: Embeddings → Stored in ChromaDB with metadata
- Step 5: ChromaDB Ready → Vector store ready for similarity queries

4.2 Query Lifecycle

- 1. User Query: String input via CLI
- 2. input_node: Trim/validate query → pass to graph
- 3. retrieve_node: Embedding search in ChromaDB → retrieve top-4
- 4. generate_node: Build prompt with context → call LLM
- 5. route_node: Evaluate escalation criteria → set escalate flag
- 6. answer_node OR escalate_node: Package answer OR create HITL ticket
- 7. Output: Print final answer (with ticket ID if escalated)

5. Technology Choices

Technology	Role	Why Chosen
LangChain	Loading, chunking	Industry standard.
ChromaDB	Vector database	Zero-setup, local persistence.
HuggingFace	Embedding model	Runs on CPU, no API cost.

LangGraph	Workflow	First-class state management.
OpenAI GPT-3.5	Generation	Affordable, high quality.
Python + Dotenv	App layer	Universal ecosystem.

6. Scalability Considerations

Large Documents: Chunk size (500) ensures $O(N)$ linear scaling. ChromaDB handles millions of embeddings.

Increasing Query Load: Stateless queries allow horizontal scaling via multiple workers.

Latency: Total P95: ~2.5s. Optimized use of async calls and streaming.

Multiple Knowledge Bases: Multi-tenant support via collection naming.

LLM Cost: GPT-3.5 at ~\$0.001/query. Cost reduction via local cache or local models.